Maze Solver Project 6

Analytical Report

FOKOU TOUO Gilles Adrien

Student ID: FOK0003

April 21, 2026

1 Problem Analysis and Algorithm Design

The objective of this project is to implement a pathfinding system that navigates a grid-based maze. The grid consists of traversable paths (0), walls (1), a starting point (2), and an exit (3).

1.1 Algorithm Choice: Backtracking (DFS)

A **Backtracking algorithm** based on **Depth-First Search (DFS)** was chosen. DFS dives deep into a single path until it finds the exit or hits a dead end, at which point it "backtracks" to explore alternative routes.

Justification:

- **Efficiency:** Highly effective for finding a valid path in a discrete grid.
- **Recursive Logic:** The call stack manages the path state implicitly.
- **State Management:** Marking visited cells with the value '4' prevents infinite loops and ensures every reachable path is explored if necessary.

2 Data Structures

The implementation relies on efficient memory management to handle large grids:

- `std::vector<std::vector<int>>`: A 2D vector allows for $O(1)$ constant-time access to any cell, which is essential for rapid movement checks.
- **Implicit Recursion Stack**: Manages the path history without needing an external stack structure.
- **ANSI Escape Codes**: Used for colored terminal output to provide a clear visual distinction between the solution path and the maze walls.

3 Asymptotic Complexity

Let V represent the total number of cells ($V = Rows \times Columns$).

- **Time Complexity:** $O(V)$. In the worst-case scenario (no path or exit at the end), every cell is visited exactly once.
- **Space Complexity:** $O(V)$. This includes the grid storage and the recursive call stack, which could reach depth V in a winding maze.

4 Testing and Verification

The solution was verified by running the solver against complex maze configurations to ensure logic handles various dead-end scenarios.

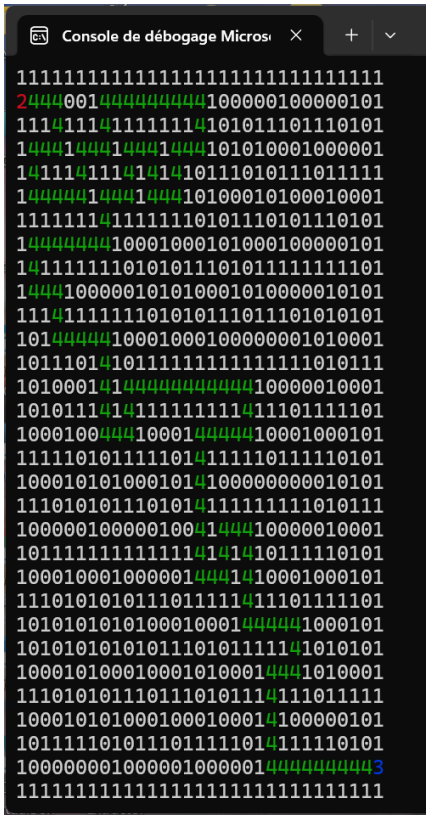


Figure 1: Successful pathfinding result.

- **Visual Debugging:** As demonstrated in Figure 1, the solver successfully identified the path from the entrance (2) to the exit (3).
- **Boundary Validation:** The `isValid` function ensures the cursor never leaves the array bounds or attempts to traverse a wall.

5 Research and Resources

The implementation logic for the recursive Backtracking and DFS strategy was informed by the following technical resources:

- `https://en.wikipedia.org/wiki/Maze-solving_algorithm`
- `https://www.geeksforgeeks.org/rat-in-a-maze-backtracking-2/`
- `https://stackoverflow.com/questions/9191428/maze-solving-algorithm-in-c`
- `https://rosettacode.org/wiki/Maze_solving`